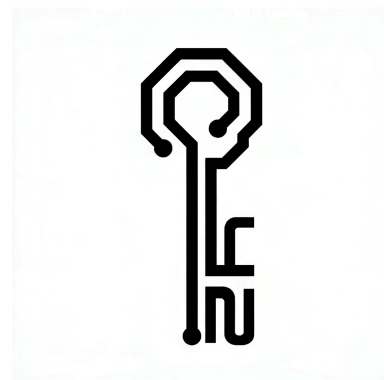


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 3
«Реализация алгоритма с использованием технологии OpenMP»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Анализ последовательного алгоритма	3
3.1. Описание	3
3.2. Блок-схема	4
4. Анализ параллельного алгоритма	5
4.1. Описание	5
4.2. Оценка параметров	5
4.3. Директивы OpenMP	5
5. Результаты экспериментов	6
5.1. Случайные массивы	6
5.2. Упорядоченные массивы	8
5.2.1. Полностью отсортированные массивы	8
5.2.2. Частично отсортированные массивы	8
5.2.3. Локально отсортированные массивы	9
5.2.4. Сравнение упорядоченных массивов	10
5.3. Дополнительное задание. Различные значения delim	11
6. Заключение	13
7. Приложение	15
7.1. Программные коды	15
7.2. Таблицы	21

1. Цель работы

Приобрести навыки разработки параллельной программы «с нуля».

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (_OPENMP = 202011).

3. Анализ последовательного алгоритма

3.1. Описание

1. Инициализируется массив из $size$ элементов, подлежащий сортировке.
2. Задаётся начальный интервал $s = size / 2$ для алгоритма Шелла.
3. Пока значение s больше нуля, выполняются следующие действия:
 - a. Массив разбивается на s последовательностей с начальными индексами $offset = 0, 1, \dots, s - 1$.
 - b. Для каждой последовательности применяется сортировка вставками:
 - i. Перебор элементов последовательности с шагом s .
 - ii. Сравнение текущего элемента с предыдущими элементами, отстоящими на шаг s .
 - iii. При обнаружении элемента, большего текущего, производится их перестановка.
4. После завершения обработки всех последовательностей интервал s уменьшается в два раза ($s /= 2$), и выполнение цикла повторяется.
5. Алгоритм завершается, когда s становится равным нулю, и массив считается полностью упорядоченным.

3.2. Блок-схема

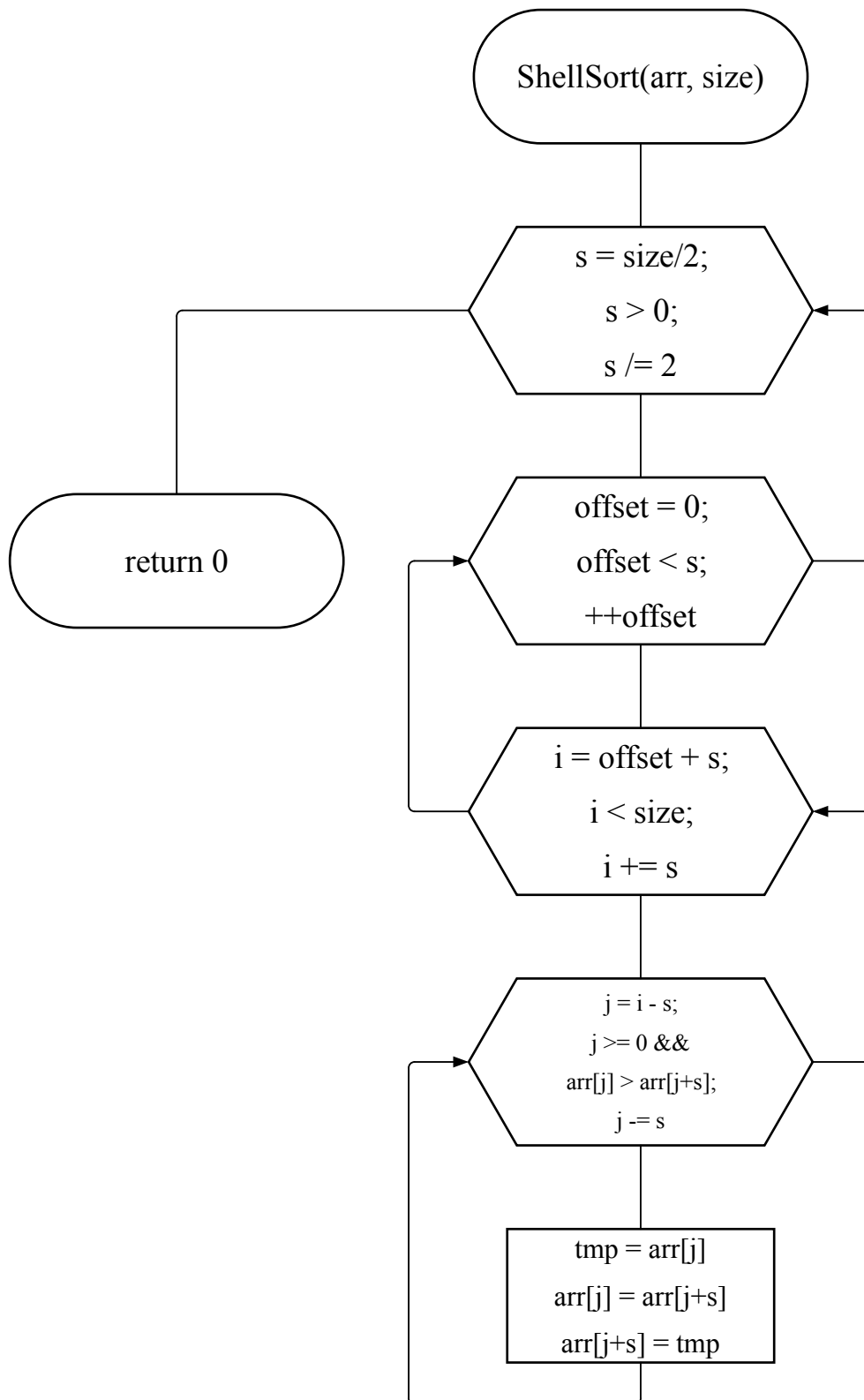


Рисунок 1 – Блок-схема алгоритма, описанного в п. 3.1

4. Анализ параллельного алгоритма

4.1. Описание

4.2. Оценка параметров

Ускорение: $S_p = \frac{T_1}{T_p}$, где T_1 – время работы в линейном режиме (1 поток), T_p – время работы при p потоках.

Эффективность: $E_p = \frac{S_p}{p}$, где S_p - ускорение, p – количество задействованных потоков.

4.3. Директивы OpenMP

#pragma omp parallel for num_threads(t) shared(array, size, s)

Регион общего доступа к array размера size (параллельная обработка независимых подмассивов с шагом s). Каждый поток выполняет локальные вставки, изменения в array распределены по offset, гонок данных нет.

5. Результаты экспериментов

Каждая точка на графике была получена путем повторения на 100 сгенерированных массивах размерами 1 млн. элементов. Массивы были одинаковы для всех точек графика

5.1. Случайные массивы

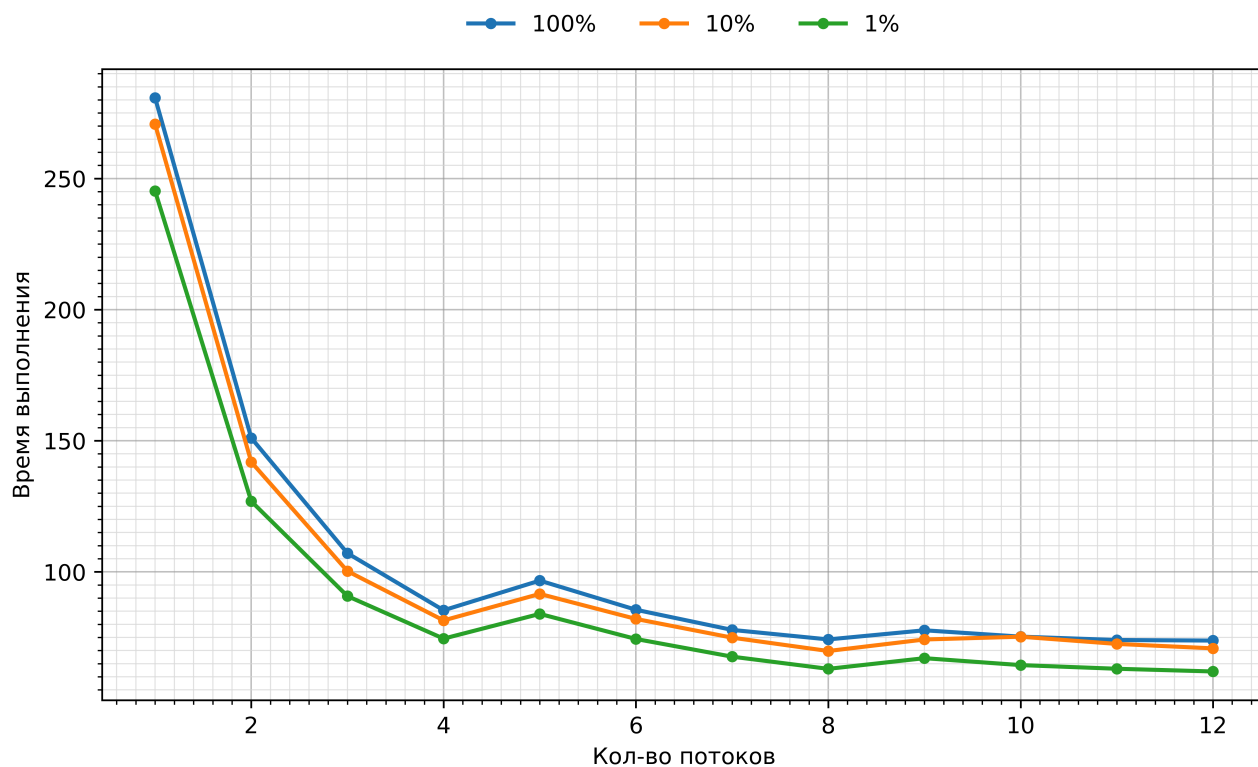


Рисунок 2 – График зависимости времени выполнения сортировки от количества потоков и уникального данных в массивах (% от размера массива)

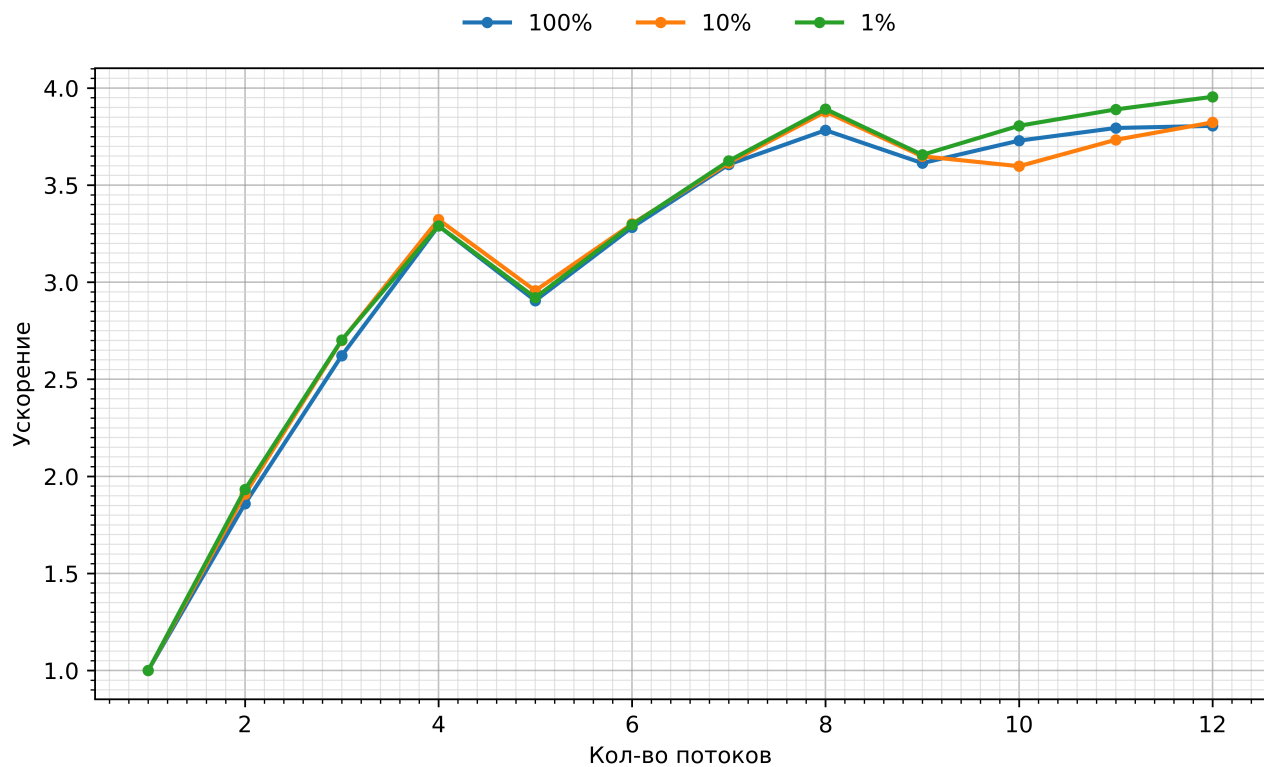


Рисунок 3 – График зависимости времени ускорения от количества потоков и уникальнй данных в массивах (% от размера массива)

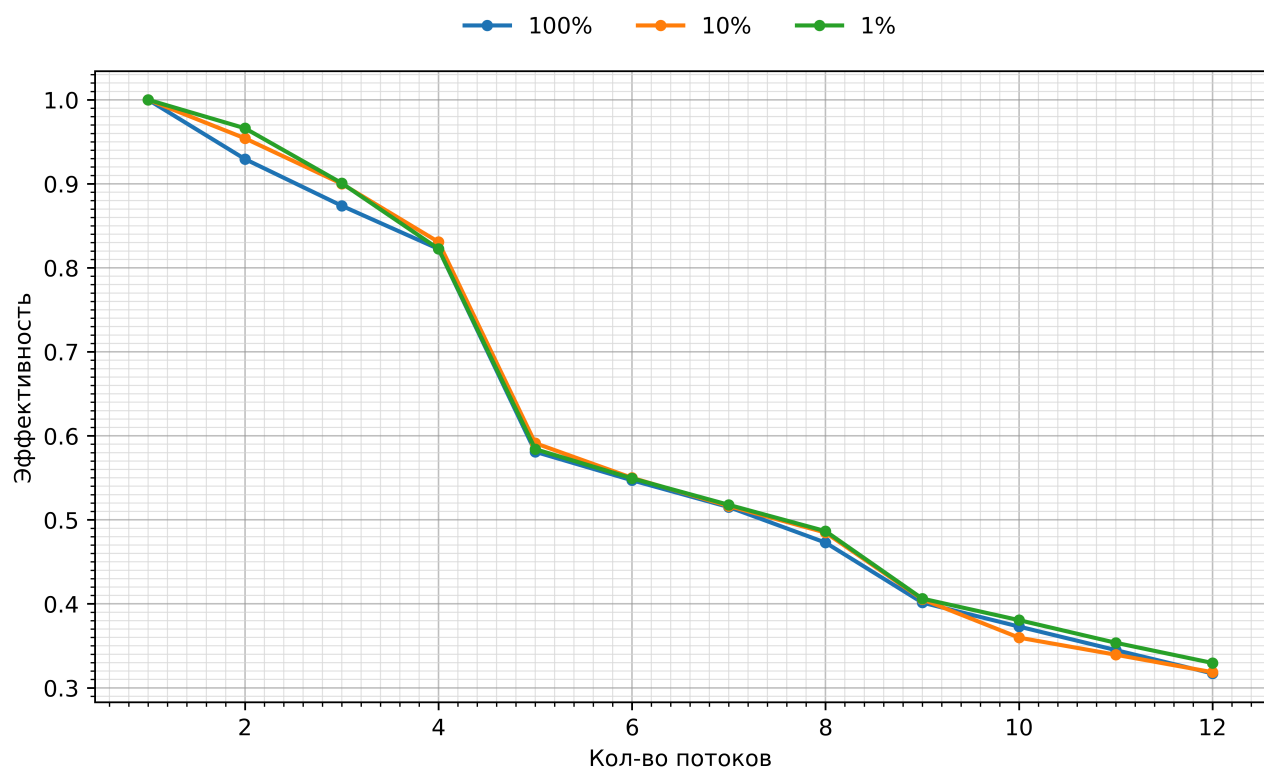


Рисунок 4 – График зависимости времени эффективности от количества потоков и уникальнй данных в массивах (% от размера массива)

5.2. Упорядоченные массивы

Следующие массивы генерировались так: сначала каждый элемент массива заполнялся числом, равным индексу элемента в массиве. Потом массив изменялся по различным правилам.

5.2.1. Полностью отсортированные массивы

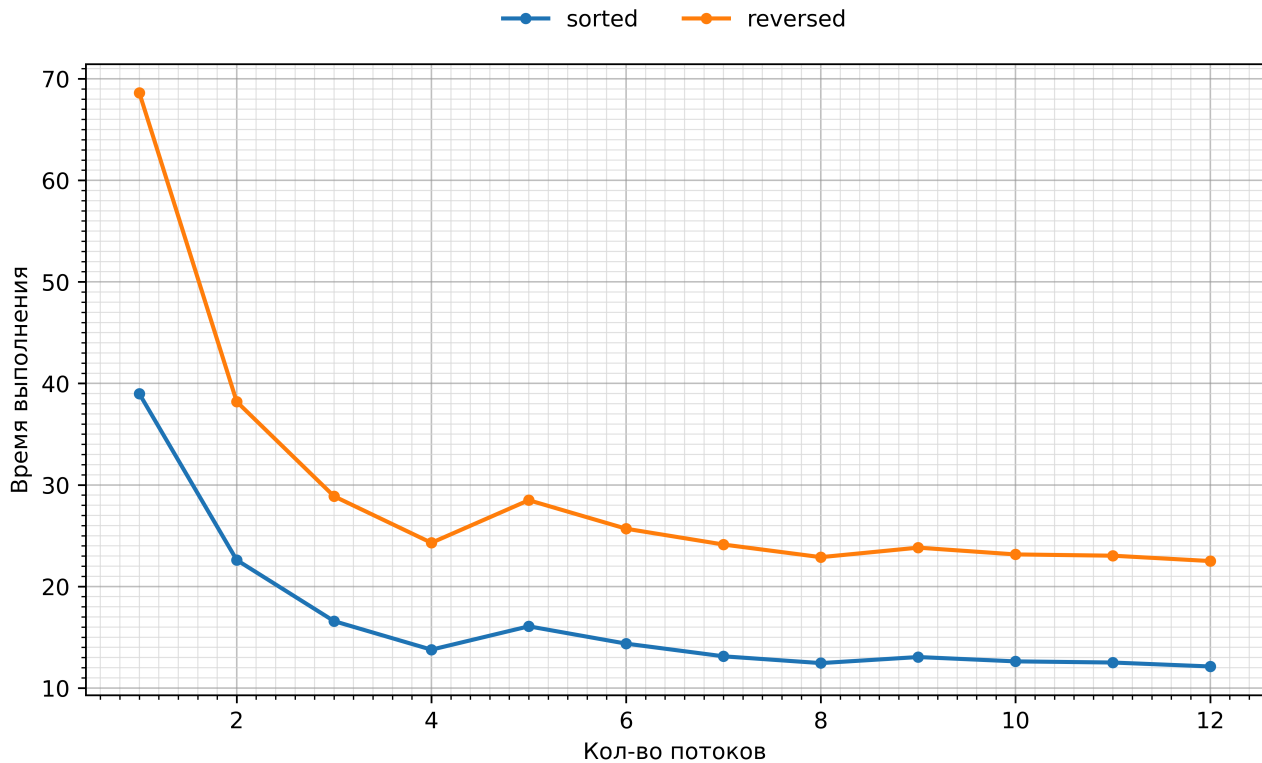


Рисунок 5 – График зависимости времени выполнения сортировки от количества потоков, на полностью отсортированных массивах (в прямом или обратном порядке)

5.2.2. Частично отсортированные массивы

Уровень отсортированности определён как доля элементов, изначально находящихся на своих позициях, что обратно пропорционально количеству случайных обменов, выполненных при генерации массива

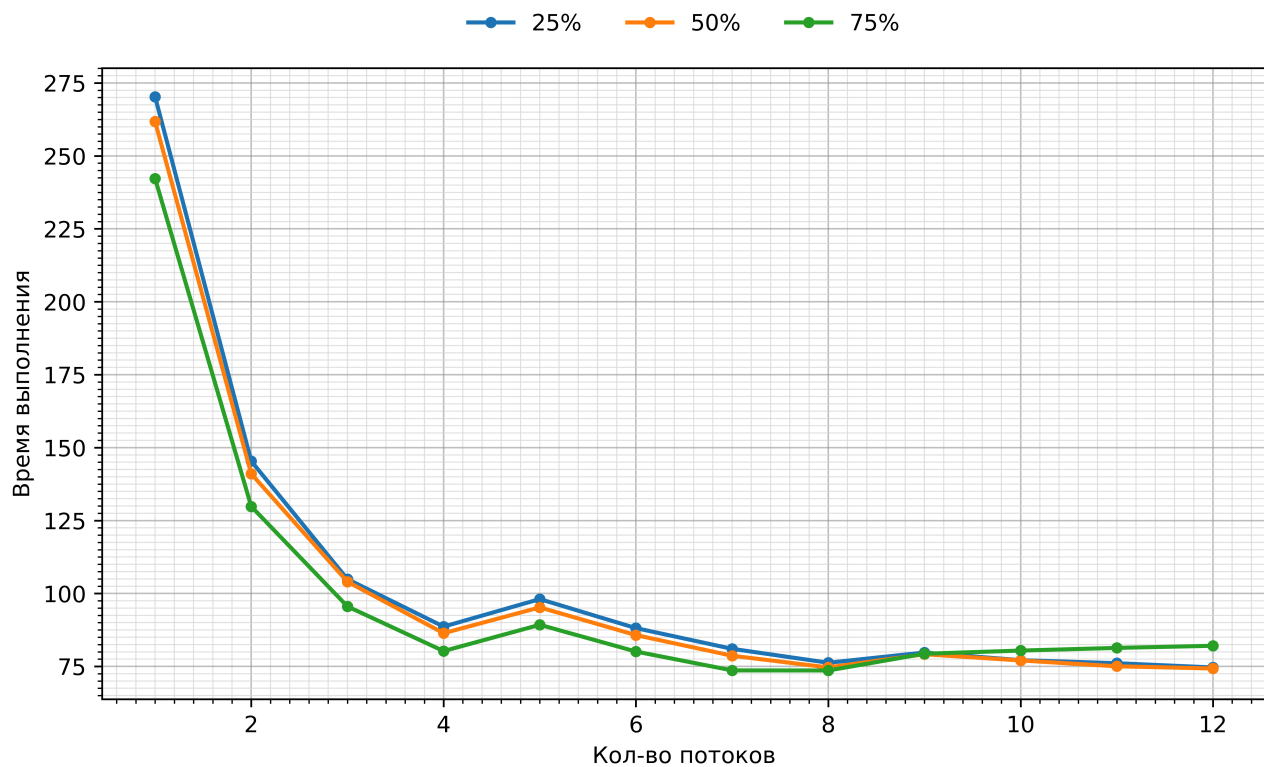


Рисунок 6 – График зависимости времени выполнения сортировки от количества потоков, на частично отсортированных массивах. Каждая кривая соответствует уровню отсортированности массива.

5.2.3. Локально отсортированные массивы

Массивы делятся на блоки, на которых они полностью отсортированы

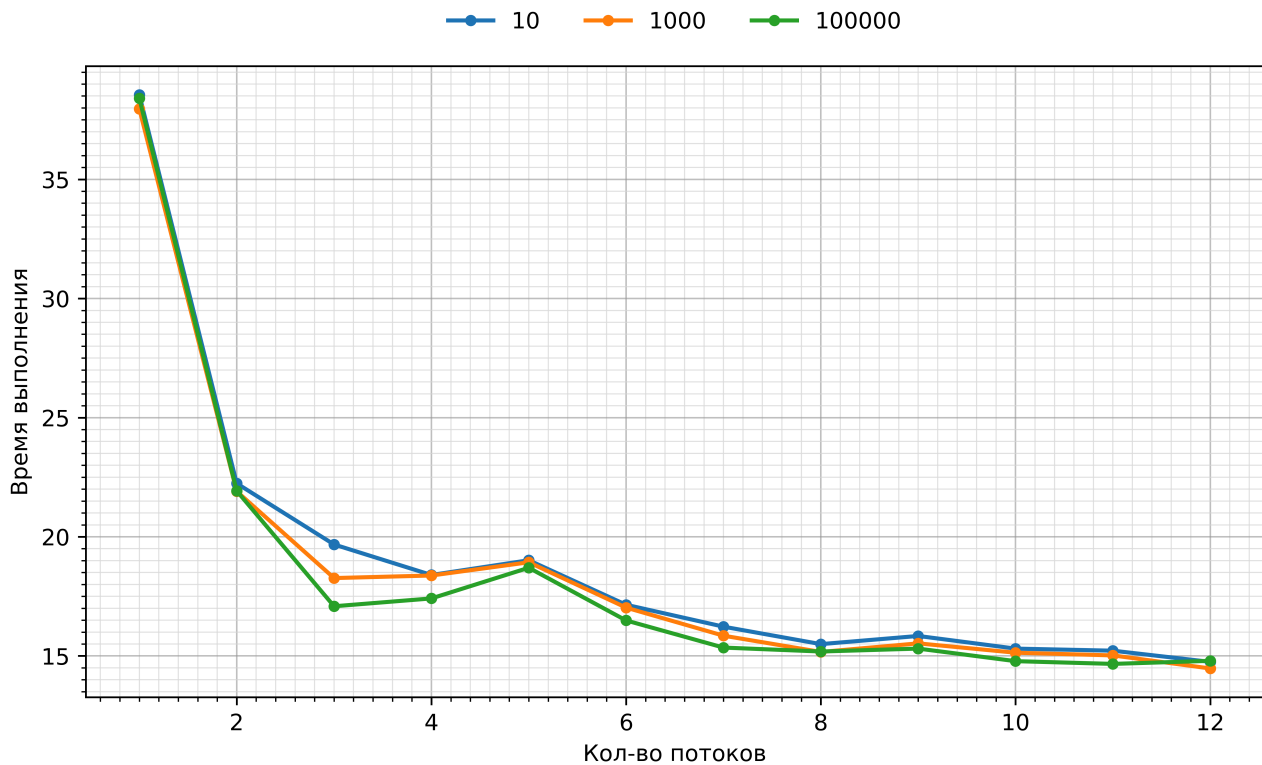


Рисунок 7 – График зависимости времени выполнения сортировки от количества потоков, на локально отсортированных массивах. Каждая кривая соответствует размеру блока.

5.2.4. Сравнение упорядоченных массивов

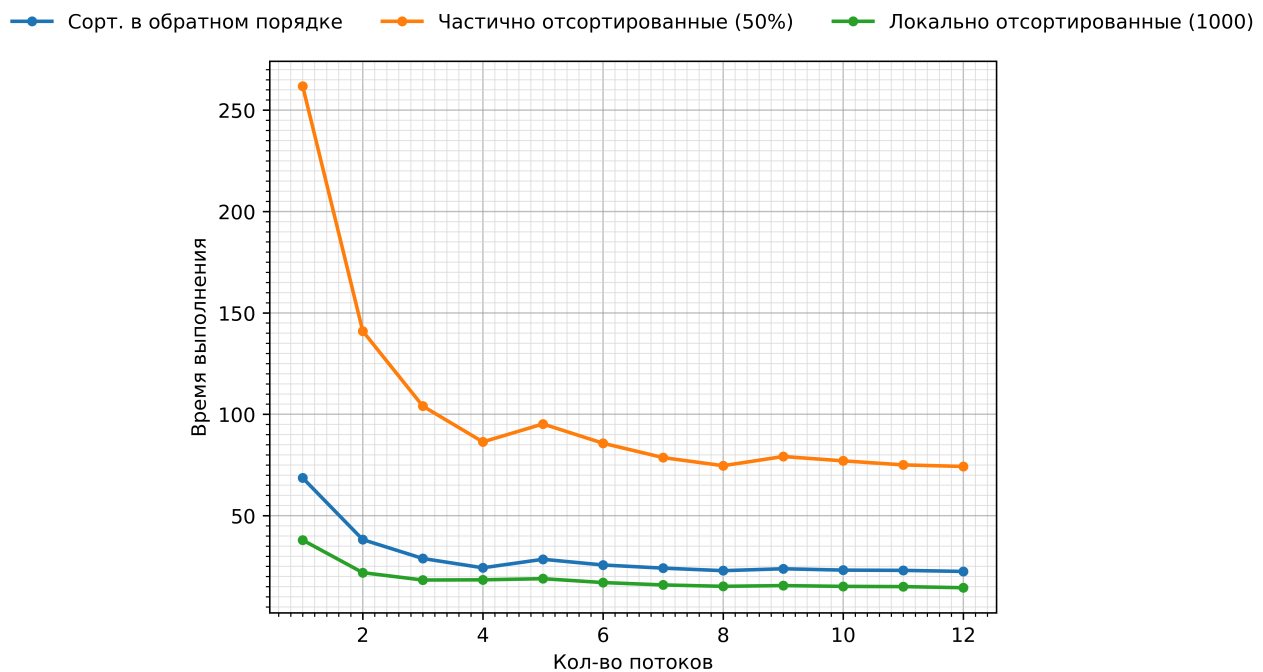


Рисунок 8 – График зависимости времени выполнения сортировки от количества потоков, на разных видах упорядоченных массивов.

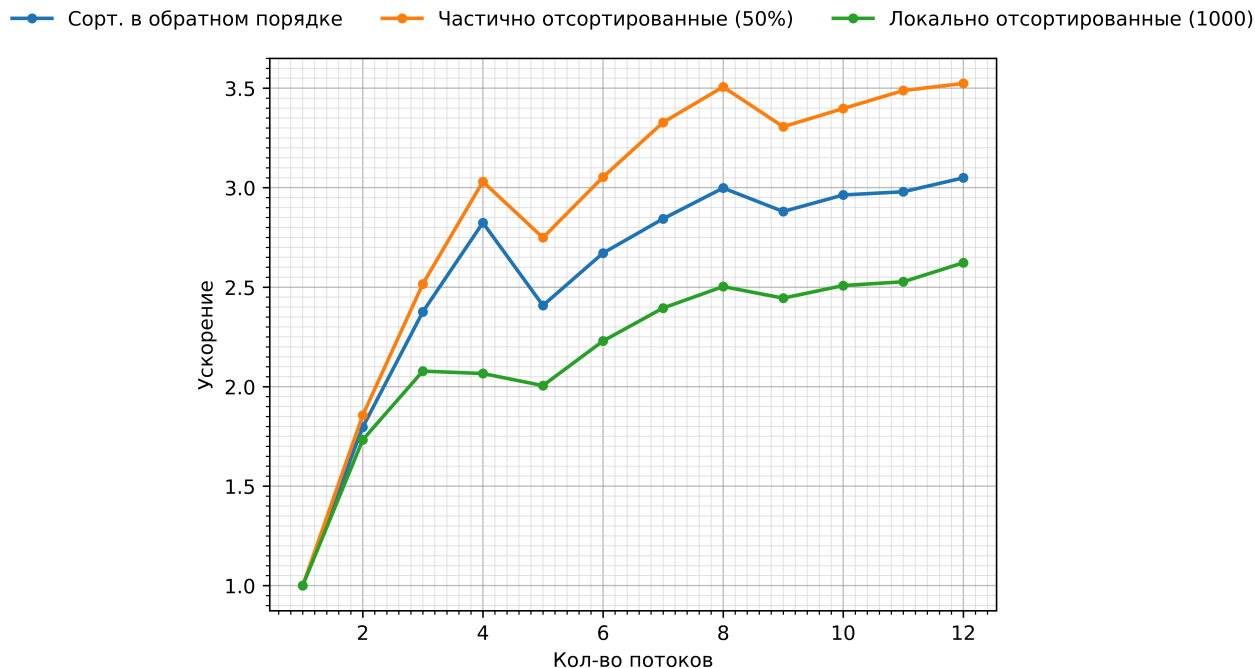


Рисунок 9 – График зависимости ускорения сортировки от количества потоков, на разных видах упорядоченных массивов.

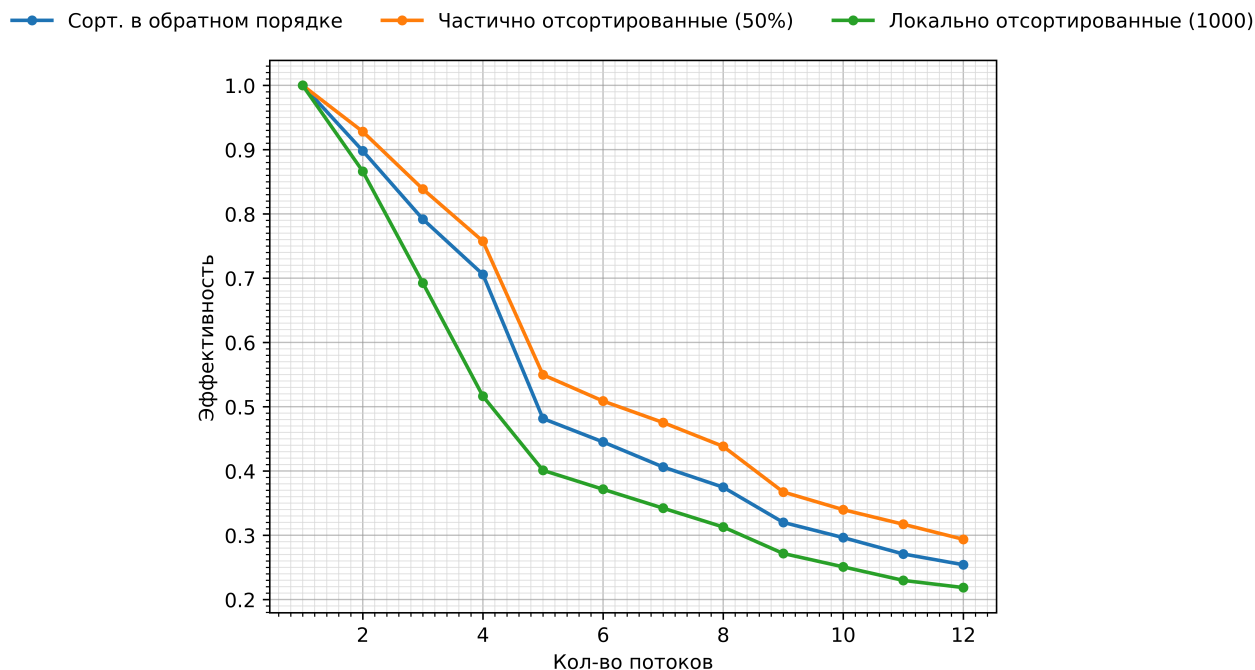


Рисунок 10 – График зависимости эффективности сортировки от количества потоков, на разных видах упорядоченных массивов.

5.3. Дополнительное задание. Различные значения `delim`

В качестве дополнительного задания алгоритм был модифицирован. Теперь он принимает значение `delim`, которое определяет скорость уменьшения шага между сортируемыми элементами.

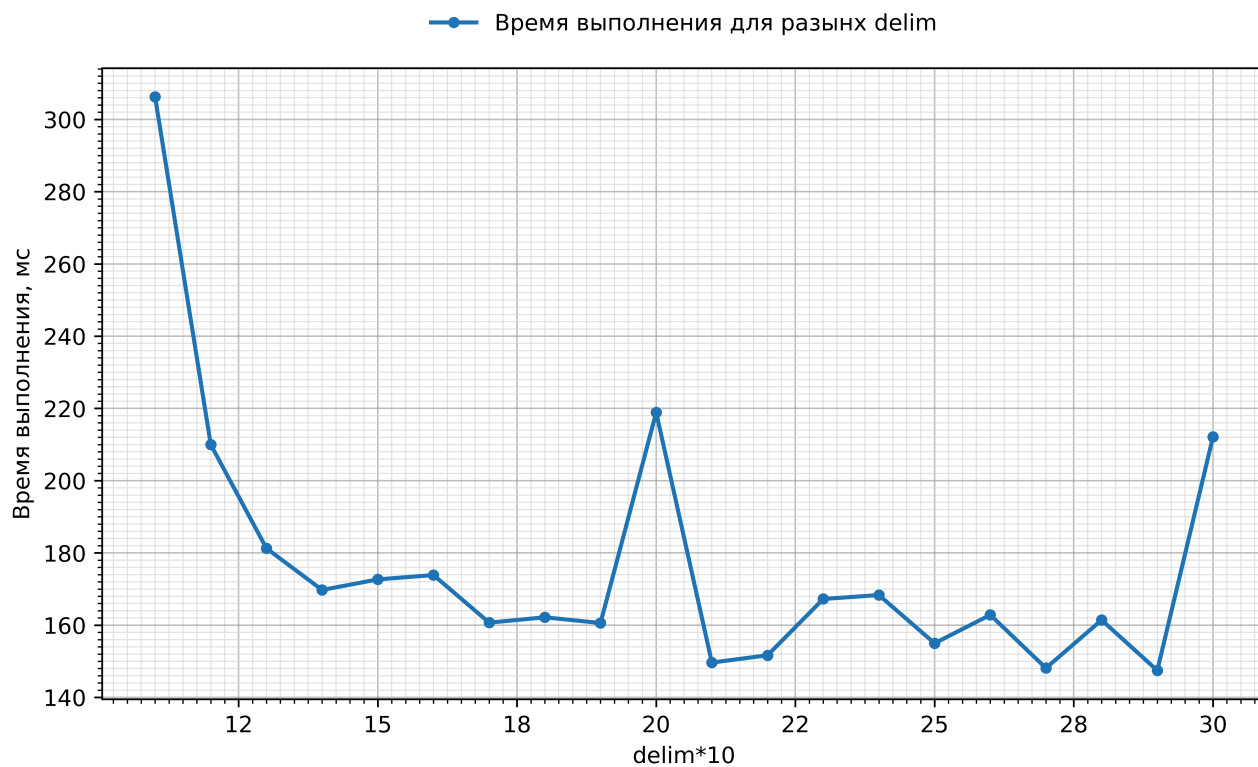


Рисунок 11 – График зависимости времени сортировки от значения delim. 8 потоков, 10^6 случайных элементов



Рисунок 12 – График зависимости времени сортировки от значения delim. 1 поток, 10^6 случайных элементов

6. Заключение

В ходе выполнения лабораторной работы были получены практические навыки разработки параллельных программ «с нуля», а также выявления и использования ресурса параллелизма в исходном последовательном алгоритме с применением технологии OpenMP.

Был рассмотрен алгоритм сортировки Шелла. Выявлена область программы, где возможно применение параллельных вычислений, и реализован соответствующий алгоритм.

Для реализации алгоритма был произведен анализ производительности на разных видах массивов. Результаты эксперимента подтвердили специфику алгоритма: частично отсортированные данные обрабатываются медленнее отсортированных (полностью или локально), но с примерно той же скоростью, что и случайно сгенерированные данные. Локально отсортированные данные сортируются очень быстро, как в случае с полностью отсортированным массивом. Размер блока локально отсортированных данных не влияет на время сортировки всего массива. Количество дубликатов в случайных данных ускоряет время сортировки массивов.

Аналогично предыдущим лабораторным, на графиках наблюдаются резкое падение ускорения после достижения 4,8 ядер. Это связано с устройством процессоров Apple M-серии (подробнее см. заключение лабораторной работы №1).

Таким образом, цель работы – построение и исследование эффективности параллельного алгоритма – была успешно достигнута.

Дополнительное задание. Алгоритм был модифицирован так, чтобы можно было устанавливать переменное значение шага между сортируемыми элементами. Был проведен эксперимент, в котором варьировался параметр `delim` (скорость уменьшения шага) и измерялось время выполнения параллельной реализации (для 8 потоков и для 1 потока, 10^6 случайных элементов). По результатам наблюдается непостоянное поведение времени: для некоторых значений `delim` время меняется плавно, а при точных целых значениях (в приведённых данных — `delim = 2.0` и `delim = 3.0`, соответствующие колонке `delim*10 = 20` и `30`) возникает резкое увеличение времени выполнения.

Как можно видеть из графиков, скачки не связаны с параллелизмом алгоритма, так как для 1 и для 8 потока наблюдаются скачки на одних и тех же значениях. Это может быть связано особенностями алгоритма сортировки: для целых значений `delim` формируются другие значения `s`, приводящие к существенно большему количеству сравнений и сдвигов.

7. Приложение

7.1. Программные коды

```
#include <iostream>

void shell_sort_parallel(int *array, int size, int threads, double delim = 2.0) {
    for (int s = size / delim; s > 0; s /= delim) {
        #pragma omp parallel for num_threads(threads) shared(array, size, s) default(none)
        for (int offset = 0; offset < s; ++offset) {
            for (int i = offset + s; i < size; i += s) {
                for (int j = i - s; j >= 0 && array[j] > array[j + s]; j -= s) {
                    int temp = array[j];
                    array[j] = array[j + s];
                    array[j + s] = temp;
                }
            }
        }
    }
}

//TODO: s from 1 to 3, step 0.1
```

Файл 1 – Функция сортировки Шелла

```
#include <vector>
#include <memory>
#include <random>
#include <algorithm>

class TMyGenerator {
public:
    TMyGenerator(int arrSize) : ArrSize_(arrSize), Rng_(Rd_()) {}

    std::unique_ptr<int32_t[]> GenPartiallySorted(double sortedPart, bool reversed) {
        auto arr = std::make_unique<int32_t[]>(ArrSize_);

        int sorted_count = static_cast<int>(ArrSize_ * sortedPart);
        if (sorted_count < 0) sorted_count = 0;
        if (sorted_count > ArrSize_) sorted_count = ArrSize_;

        std::iota(arr.get(), arr.get() + ArrSize_, 0);

        int shuffle_count = static_cast<int>((1.0 - sortedPart) * ArrSize_ / 2);
        std::uniform_int_distribution<int> dist(0, ArrSize_ - 1);

        for (int k = 0; k < shuffle_count; ++k) {
            int i = dist(Rng_);
            int j = dist(Rng_);
            std::swap(arr[i], arr[j]);
        }

        if (reversed) {
            std::reverse(arr.get(), arr.get() + ArrSize_);
        }

        return arr;
    }
}
```

```

std::unique_ptr<int32_t[]> GenLocallySorted(int32_t blockSize) {
    std::unique_ptr<int32_t[]> arr = std::make_unique<int[]>(ArrSize_);

    std::iota(arr.get(), arr.get() + ArrSize_, 1);

    for (int32_t i = 0; i < ArrSize_; i += blockSize) {
        size_t end = std::min(i + blockSize, ArrSize_);
        std::sort(arr.get() + i, arr.get() + end);
    }
    return arr;
}

std::unique_ptr<int32_t[]> GenRandom(double uniquePart) {
    std::uniform_int_distribution<int32_t> dist(0, ArrSize_ * uniquePart);

    std::unique_ptr<int32_t[]> arr = std::make_unique<int[]>(ArrSize_);

    for (int i = 0; i < ArrSize_; ++i){
        arr[i] = dist(Rng_);
    }
    return arr;
}

private:
    int ArrSize_;
    std::random_device Rd_;
    std::mt19937_64 Rng_;
};

```

Файл 2 – Генератор случайных массивов

```

#include "gen.h"
#include "shell.h"
#include <fstream>
#include <vector>
#include <iostream>
#include <optional>
#include <omp.h>
#include <filesystem>

namespace fs = std::filesystem;

using TResults = std::vector<std::vector<double>>>;
using TResultLine = std::vector<double>;
using TData = std::vector<std::vector<std::unique_ptr<int32_t[]>>>>;
using TSeries = std::vector<std::unique_ptr<int32_t[]>>>;

int32_t elems = pow(10, 6);
int32_t arrays = 100;
int threads = 12;

TData    randomData, // full, 1%
         fullySortedData, // sorted, reverted
         partiallySortedData, // 25, 50, 75

```

```

        locallySortedData; // 10, 1000, 100000

void prepareData(
    double random_coef,
    std::optional<bool> is_reverse,
    double part_coef,
    int32_t block_size
)
{
    static TMyGenerator gen(elems);
    TSeries seriesRandom, seriesFully, seriesPartial, seriesLocally;
    for (int i = 0; i < arrays; ++i) {
        std::cout << i << std::endl;
        seriesRandom.push_back(std::move(gen.GenRandom(random_coef)));
        if (is_reverse.has_value()) {
            seriesFully.push_back(std::move(gen.GenPartiallySorted(1, is_reverse.value())));
        }
        seriesPartial.push_back(std::move(gen.GenPartiallySorted(part_coef, false)));
        seriesLocally.push_back(std::move(gen.GenLocallySorted(block_size)));
    }
    if (is_reverse.has_value()) {
        fullySortedData.push_back(std::move(seriesFully));
    }
    randomData.push_back(std::move(seriesRandom));
    partiallySortedData.push_back(std::move(seriesPartial));
    locallySortedData.push_back(std::move(seriesLocally));
}

TResults process(TData& data) {
    TResults results;
    static std::unique_ptr<int32_t[]> copy = std::make_unique<int32_t[]>(elems);
    for (const auto& series : data){
        TResultLine line;
        for (int t=1; t <= threads; ++t) {
            double sum = 0;
            for (const auto& array : series) {
                std::copy(array.get(), array.get() + elems, copy.get());
                double s = omp_get_wtime();
                shell_sort_parallel(copy.get(), elems, t);
                double e = omp_get_wtime();
                sum += e - s;
            }
            sum /= arrays;
            sum *= 1000; // to ms
            line.push_back(sum);
            std::cout << "Finished " << t << std::endl;
        }
        results.push_back(line);
    }
    return results;
}

void prepare_headers(
    std::ofstream& fileRand,
    std::ofstream& fileRandSpeed,
    std::ofstream& fileRandEff,
    std::ofstream& fileFully,

```

```

std::ofstream& filePartial,
std::ofstream& fileLocal,
std::ofstream& fileTogether,
std::ofstream& fileTogetherSpeed,
std::ofstream& fileTogetherEff
) {
    fileRand << "3\n" << "100%\n" << "10%\n" << "1%\n" << "Кол-во потоков\n" << "Время выполнения\n"
    << "none\n";
    fileRandSpeed << "3\n" << "100%\n" << "10%\n" << "1%\n" << "Кол-во потоков\n" << "Ускорение\n"
    << "none\n";
    fileRandEff << "3\n" << "100%\n" << "10%\n" << "1%\n" << "Кол-во потоков\n" << "Эффективность\n"
    << "none\n";

    fileFully << "2\n" << "sorted\n" << "reversed\n" << "Кол-во потоков\n" << "Время выполнения\n"
    << "none\n";
    filePartial << "3\n" << "25%\n" << "50%\n" << "75%\n" << "Кол-во потоков\n" << "Время
    выполнения\n" << "none\n";
    fileLocal << "3\n" << "10\n" << "1000\n" << "100000\n" << "Кол-во потоков\n" << "Время
    выполнения\n" << "none\n";

    fileTogether << "3\n" << "Сорт. в обратном порядке\n" << "Частично отсортированные (50%)\n"
    << "Локально отсортированные (1000)\n" << "Кол-во потоков\n" << "Время выполнения\n" << "none\n";
    fileTogetherSpeed << "3\n" << "Сорт. в обратном порядке\n" << "Частично отсортированные (50%)\n"
    << "Локально отсортированные (1000)\n" << "Кол-во потоков\n" << "Ускорение\n" << "none\n";
    fileTogetherEff << "3\n" << "Сорт. в обратном порядке\n" << "Частично отсортированные (50%)\n"
    << "Локально отсортированные (1000)\n" << "Кол-во потоков\n" << "Эффективность\n" << "none\n";
}

void test_delim_graph()
{
    TData& data = randomData;

    static std::unique_ptr<int32_t[]> copy = std::make_unique<int32_t[]>(elems);
    const double eps = 1e-9;

    struct GraphFile {
        std::string filename;
        int threads;
    };

    std::vector<GraphFile> files = {
        {"results/delim_graph_1thread", 1},
        {"results/delim_graph_8threads", 8}
    };

    for (const auto& gf : files) {
        std::ofstream file(gf.filename);
        file << "1\n";
        file << "Время выполнения для разных delim\n";
        file << "delim*10\n";
        file << "Время выполнения, мс\n";
        file << "none\n";

        double delim = 1.1;
        while (delim <= 3.0 + eps) {
            double sum = 0;

```

```

        for (const auto& series : data) {
            for (const auto& array : series) {
                std::copy(array.get(), array.get() + elems, copy.get());
                double s = omp_get_wtime();
                shell_sort_parallel(copy.get(), elems, gf.threads, delim);
                double e = omp_get_wtime();
                sum += e - s;
            }
        }

        sum /= arrays;
        sum *= 1000; // ms

        file << delim*10 << " " << sum << "\n";
        std::cout << "Finished delim=" << delim << " threads=" << gf.threads << " time=" <<
sum << "ms\n";

        delim += 0.1;
        delim = std::round(delim * 10.0) / 10.0;
    }
}

int main() {
    std::cout << "preparing data" << std::endl;
    prepareData(1, false, 0.25, 10);
    prepareData(0.1, true, 0.50, 1000);
    prepareData(0.01, std::nullopt, 0.75, 100000);

    std::cout << "calculating" << std::endl;
    TResults resultsRandom = process(randomData);
    TResults resultsFullySorted = process(fullySortedData);
    TResults resultsPartiallySorted = process(partiallySortedData);
    TResults resultsLocallySorted = process(locallySortedData);

    std::ofstream fileRand("results/random");
    std::ofstream fileRandSpeed("results/rand_speed");
    std::ofstream fileRandEff("results/rand_eff");

    std::ofstream fileFully("results/fully");
    std::ofstream filePartial("results/part");
    std::ofstream fileLocal("results/local");

    std::ofstream fileTogether("results/together");
    std::ofstream fileTogetherSpeed("results/together_speed");
    std::ofstream fileTogetherEff("results/together_eff");
    prepare_headers(fileRand, fileRandSpeed, fileRandEff, fileFully, filePartial, fileLocal,
fileTogether, fileTogetherSpeed, fileTogetherEff);
    for (int t = 0; t < threads; ++t) {
        fileRand << t + 1 << " ";
        fileRandSpeed << t + 1 << " ";
        fileRandEff << t + 1 << " ";
        fileFully << t + 1 << " ";
        filePartial << t + 1 << " ";
        fileLocal << t + 1 << " ";
    }
}

```

```

fileTogether << t + 1 << " ";
fileTogetherSpeed << t + 1 << " ";
fileTogetherEff << t + 1 << " ";

for (const auto& arr : resultsRandom) {
    fileRand << arr[t] << " ";
    fileRandSpeed << arr[0] / arr[t] << " ";
    fileRandEff << arr[0] / arr[t] / (t + 1) << " ";
}
fileRand << "\n";
fileRandSpeed << "\n";
fileRandEff << "\n";

for (const auto& arr : resultsFullySorted) {
    fileFully << arr[t] << " ";
}
fileFully << "\n";
for (const auto& arr : resultsPartiallySorted) {
    filePartial << arr[t] << " ";
}
filePartial << "\n";
for (const auto& arr : resultsLocallySorted) {
    fileLocal << arr[t] << " ";
}
fileLocal << "\n";

fileTogether << resultsFullySorted[1][t] << " " << resultsPartiallySorted[1][t] << " " <<
resultsLocallySorted[1][t] << "\n";
fileTogetherSpeed << resultsFullySorted[1][0] / resultsFullySorted[1][t]
    << " " << resultsPartiallySorted[1][0] / resultsPartiallySorted[1][t]
    << " " << resultsLocallySorted[1][0] / resultsLocallySorted[1][t] << "\n";
fileTogetherEff << resultsFullySorted[1][0] / resultsFullySorted[1][t] / (t + 1)
    << " " << resultsPartiallySorted[1][0] / resultsPartiallySorted[1][t] / (t + 1)
    << " " << resultsLocallySorted[1][0] / resultsLocallySorted[1][t] / (t + 1)
    << "\n";
}

test_delim_graph();
}

```

Файл 3 – Замер времени для сгенерированных массивов

7.2. Таблицы

Таблица 1 – Данные о времени работы в зависимости от кол-ва дубликатов в случайных данных

Кол-во потоков	100%	10%	1%
1	280.782	270.746	245.225
2	151.049	141.878	126.914
3	107.105	100.271	90.7429
4	85.3446	81.4755	74.5284
5	96.6586	91.5828	83.9629
6	85.5388	82.0435	74.3948
7	77.8562	74.8977	67.6506
8	74.2254	69.8122	63.0066
9	77.7081	74.2039	67.0747
10	75.2949	75.2612	64.4345
11	74.0014	72.5179	63.0406
12	73.7884	70.8183	62.0048

Таблица 2 – Данные о времени работы в зависимости от порядка отсортированности

Кол-во потоков	sorted	reversed
1	38.9842	68.62
2	22.608	38.2013
3	16.587	28.8845
4	13.7754	24.3
5	16.076	28.4974
6	14.371	25.6886
7	13.1214	24.1313
8	12.4547	22.8874
9	13.0508	23.8244
10	12.6245	23.1565
11	12.5101	23.0315
12	12.1216	22.5037

Таблица 3 – Данные о времени работы в зависимости от отсортированности массива

Кол-во потоков	25%	50%	75%
1	270.261	261.757	242.233
2	145.362	141.02	129.825
3	104.91	104.032	95.5595
4	88.6391	86.3804	80.2531
5	98.0719	95.2294	89.2618
6	88.1242	85.7325	80.0929
7	81.0267	78.654	73.629
8	76.2513	74.6484	73.6048
9	79.7629	79.1707	79.3455
10	77.1304	77.039	80.4236
11	76.0953	75.0432	81.3406
12	74.631	74.2791	82.071

Таблица 4 – Данные о времени работы в зависимости от размера блока

Кол-во потоков	10	1000	100000
1	38.549	37.9578	38.3994
2	22.2408	21.9076	21.9266
3	19.6765	18.267	17.0837
4	18.3966	18.3745	17.4125
5	19.0142	18.9301	18.6997
6	17.1465	17.0266	16.4881
7	16.2269	15.8485	15.3497
8	15.4946	15.1655	15.1852
9	15.8371	15.5267	15.3073
10	15.3064	15.1344	14.7799
11	15.2192	15.021	14.6647
12	14.7488	14.4728	14.7937

Таблица 5 – Данные о времени работы в зависимости от delim, 8 потоков

delim*10	Время выполнения для разных delim
11	306.26
12	209.965
13	181.262
14	169.742
15	172.663
16	173.873
17	160.7
18	162.169
19	160.593
20	218.89
21	149.643
22	151.657
23	167.246
24	168.331
25	154.983
26	162.841
27	148.123
28	161.428
29	147.455
30	212.106

Таблица 6 – Данные о времени работы в зависимости от delim, 1 поток

delim*10	Время выполнения для разных delim
11	1081.99
12	744.841
13	635.769
14	602.7
15	603.92
16	599.08
17	596.473
18	606.552
19	607.314
20	804.323
21	590.645
22	604.536
23	614.419
24	616.537
25	621.155
26	603.512
27	597.133
28	586.908
29	571.738
30	747.584